# std::nuclear

# Содержание

1 Структуры данных ..... 3
1.1 DSU — система непересекающихся множеств ..... 3
1.2 Двумерное дерево Фенвика ..... 3
1.3 Splay-дерево с явным ключом ..... 3
1.4 Splay-дерево с неявным ключом ..... 4
1.5 Link-cut tree ..... 5
1.6 Дерево отрезков с массовыми операциями RECHECK ..... 7
1.7 Декартово дерево (явное/неявное, sum + size) RECHECK ..... 7
1.8 Ordered set/map RECHECK ..... 8
1.9 Дерево Фенвика RECHECK ..... 8
2 Строковые алгоритмы ..... 9
2.1 Динамический Ахо — Корасик ..... 9
2.2 Суффиксный автомат RECHECK ..... 10
2.3 Суффиксное дерево RECHECK ..... 10
2.4 Суффиксный массив и LCP RECHECK ..... 11
2.5 Ахо — Корасик RECHECK ..... 12
2.6 Z-функция RECHECK ..... 12
2.7 Префикс-функция RECHECK ..... 12
3 Геометрия ..... 13
3.1 Точка и векторы RECHECK ..... 13
3.2 Выпуклая оболочка Грэхема O(NlogN) RECHECK ..... 13
4 Графы ..... 13
4.1 Диниц $O(V^2E)$ RECHECK ..... 13
4.2 Min Cost Max Flow (MCMF) RECHECK ..... 14
4.3 Максимальное паросочетание (Кун с таймером) RECHECK ..... 15
4.4 HLD RECHECK ..... 15
4.5 Максимальное паросочетание (Кун, пример с main) RECHECK ..... 17
5 Теория чисел ..... 17
5.1 Расширенный алгоритм Евклида ..... 17
5.2 Китайская теорема об остатках ..... 18
5.3 Свёртка целых чисел через FFT ..... 18
5.4 Линейное решето Эратосфена O(n) RECHECK ..... 19
5.5 Проверка на простоту Миллера — Рабина RECHECK ..... 19
5.6 Длинная арифметика (ГПТ) RECHECK ..... 19
6 C++ ..... 20
6.1 Прагмы, генератор случайных чисел и замер времени (ГПТ, надо проверить) RECHECK ..... 20
7 Теоретические справки ..... 21
7.1 Формулы, обоснования и условия применения ..... 21
8 Идеи решения задач ..... 26
8.1 Повторяющиеся приёмы московских контестов ..... 26

# 1 Структуры данных

## 1.1 DSU — система непересекающихся множеств

```
/// @author OpenAI Codex
/// @brief DSU со сжатием путей и объединением по размеру.
/// @date 2026-11-07

#include <numeric>
#include <utility>
#include <vector>

// Вершины 0..n-1. Изначально каждая в отдельной компоненте.
class DSU {
    std::vector<int> parent_, size_;

public:
    explicit DSU(int n) : parent_(n), size_(n, 1) {
        std::iota(parent_.begin(), parent_.end(), 0);
    }

    int find(int v) {
        if (parent_[v] != v) parent_[v] = find(parent_[v]);
        return parent_[v];
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return false;
        if (size_[a] < size_[b]) std::swap(a, b);
        parent_[b] = a;
        size_[a] += size_[b];
        return true;
    }

    bool same(int a, int b) { return find(a) == find(b); }
    int size(int v) { return size_[find(v)]; }
};
```

## 1.2 Двумерное дерево Фенвика

```
/// @author arkadiy
template <typename T>
struct FenwickTree2D {
    int n, m;
    vector<vector<T>> t;

    /// @note n ~ x, m ~ y
    FenwickTree2D(int n, int m) : n(n), m(m), t(n + 1, vector<T>(m + 1)) {}

    void Add(int x, int y, int v) {
        x++, y++;
        for (int i = x; i <= n; i |= (i + 1)) {
            for (int j = y; j <= m; j |= (j + 1)) {
                t[i][j] += v;
            }
        }
    }

    T Sum(int x, int y) {
        T s = 0;
        for (int i = x; i > 0; i = (i & (i + 1)) - 1) {
            for (int j = y; j > 0; j = (j & (j + 1)) - 1) {
                s += t[i][j];
            }
        }
        return s;
    }

    T Sum(int x0, int y0, int x1, int y1) {
        x1++, y1++;
        return Sum(x1, y1) - Sum(x0, y1) - Sum(x1, y0) + Sum(x0, y0);
    }
};
```

## 1.3 Splay-дерево с явным ключом

```
/// @author arkadiy
namespace splay {
struct Node {
    Node *l = nullptr, *r = nullptr, *p = nullptr;
    i64 v = 0;

    Node(i64 v) : v(v) {}
};

void rotate(Node* t) {
    Node* p = t->p;
    Node* g = p->p;

    if (p->l == t) {
        p->l = t->r;
        if (p->l) p->l->p = p;
        t->r = p;
    } else {
        p->r = t->l;
        if (p->r) p->r->p = p;
        t->l = p;
    }

    p->p = t;
    t->p = g;
    if (g) {
        if (g->l == p)
            g->l = t;
        else
            g->r = t;
    }
}

Node* splay(Node* t) {
    while (t && t->p) {
        Node *p = t->p, *g = t->p->p;
        if (!g) {
            rotate(t);
        } else if ((g->l == p) ^ (p->l == t)) {
            rotate(t);
            rotate(t);
        } else {
            rotate(p);
            rotate(t);
        }
    }
    return t;
}

Node* max(Node* t) {
    while (t && t->r) {
```

```
        t = t->r;
    }
    splay(t);
    return t;
}

Node* merge(Node* t1, Node* t2) {
    if (!t1) {
        return t2;
    }
    if (!t2) {
        return t1;
    }

    t1 = max(t1);
    t1->r = t2;
    t2->p = t1;

    return t1;
}

Node* find(Node* t, i64 x) {
    if (t == nullptr) {
        return nullptr;
    }

    Node* last = t;
    while (t) {
        last = t;
        if (t->v == x) break;
        if (x < t->v) {
            t = t->l;
        } else {
            t = t->r;
        }
    }

    return splay(last);
}

pair<Node*, Node*> split(Node* t, i64 x) {
    if (!t) {
        return {nullptr, nullptr};
    }

    t = find(t, x);

    if (t->v >= x) {
        Node* l = t->l;
        t->l = nullptr;
        if (l) {
            l->p = nullptr;
        }

        return {l, t};
    } else {
        Node* r = t->r;
        t->r = nullptr;
        if (r) {
            r->p = nullptr;
        }

        return {t, r};
    }
}
}  // namespace splay
```

## 1.4 Splay-дерево с неявным ключом

```
/// @author arkadiy
// TODO: refactor :(

struct Node {
    Node* l = nullptr;
    Node* r = nullptr;
    Node* p = nullptr;

    i64 sz = 0, v = 0, mn = 0;

    bool rev = false;  // reverse (lazy prop)
    i64 to_add = 0;    // add (lazy prop)

    Node(i64 v) : v(v), mn(v), sz(1) {}
};

void push(Node* t) {
    if (t->rev) {
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= 1;
        if (t->r) t->r->rev ^= 1;
        t->rev = false;
    }

    t->mn += t->to_add;
    t->v += t->to_add;
    if (t->l) {
        t->l->to_add += t->to_add;
    }
    if (t->r) {
        t->r->to_add += t->to_add;
    }
    t->to_add = 0;
}

void pull(Node* t) {
    push(t);

    t->sz = 1 + (t->l ? t->l->sz : 0) + (t->r ? t->r->sz : 0);
    t->mn = t->v;
    if (t->l) {
        push(t->l);
        t->mn = min(t->mn, t->l->mn);
    }
    if (t->r) {
        push(t->r);
        t->mn = min(t->mn, t->r->mn);
    }
}

void rotate(Node* t) {
    Node* p = t->p;
    Node* g = p->p;

    if (g) push(g);
    push(p);
    push(t);

    if (p->l == t) {
        p->l = t->r;
        if (p->l) p->l->p = p;
        t->r = p;
    } else {
        p->r = t->l;
```

```
        if (p->r) p->r->p = p;
        t->l = p;
    }

    p->p = t;
    t->p = g;
    pull(p);
    pull(t);

    if (g) {
        if (g->l == p)
            g->l = t;
        else
            g->r = t;
        pull(g);
    }
}

Node* splay(Node* t) {
    while (t && t->p) {
        Node *p = t->p, *g = t->p->p;
        if (!g) {
            rotate(t);
        } else if ((g->l == p) ^ (p->l == t)) {
            rotate(t);
            rotate(t);
        } else {
            rotate(p);
            rotate(t);
        }
    }
    return t;
}

Node* max(Node* t) {
    if (!t) return nullptr;
    push(t);

    while (t->r) {
        t = t->r;
        push(t);
    }
    return splay(t);
}

Node* merge(Node* t1, Node* t2) {
    if (!t1) return t2;
    if (!t2) return t1;

    t1 = max(t1);
    t1->r = t2;
    t2->p = t1;
    pull(t1);

    return t1;
}

Node* get(Node* t, i64 i) {
    if (!t || i >= t->sz) {
        return nullptr;
    }

    while (t) {
        push(t);
        i64 leftsz = (t->l ? t->l->sz : 0);
        if (i < leftsz) {
            t = t->l;
        } else if (i == leftsz) {
            return splay(t);
        } else {
            i -= leftsz + 1;
            t = t->r;
        }
    }

    return nullptr;
}

pair<Node*, Node*> split(Node* t, i64 cnt) {
    if (!t) return {nullptr, nullptr};
    if (cnt >= t->sz) return {t, nullptr};

    t = get(t, cnt);

    Node *l = t->l, *r = t;
    if (l) {
        l->p = nullptr;
    }
    if (r) {
        r->l = nullptr;
        pull(r);
    }

    return {l, r};
}

// remove independent subtree (by root)
void remove(Node* t) {
    if (!t) return;
    remove(t->l);
    remove(t->r);
    delete t;
}

// create new tree with one node
Node* create(i64 v) { return new Node(v); }
```

## 1.5 Link-cut tree

```
/// @author arkadiy
/// @date 2026-10-17

// TODO: refactor + comments
namespace splay {
const int inf = 1e9;
struct Node {
    Node* l = nullptr;
    Node* r = nullptr;
    Node* p = nullptr;

    i64 sz = 0, v = 0, sm = 0;

    bool rev = false;  // reverse (lazy prop)
    i64 to_add = 0;    // add (lazy prop)

    Node(i64 v) : v(v), sm(v), sz(1) {}
};

bool is_root(Node* t) { return (!t->p || t->p->l != t && t->p->r != t); }

void push(Node* t) {
```

```
    if (t->rev) {
        swap(t->l, t->r);
        if (t->l) t->l->rev ^= 1;
        if (t->r) t->r->rev ^= 1;
        t->rev = false;
    }

    t->sm += t->to_add * t->sz;
    t->v += t->to_add;
    if (t->l) {
        t->l->to_add += t->to_add;
    }
    if (t->r) {
        t->r->to_add += t->to_add;
    }
    t->to_add = 0;
}

void pull(Node* t) {
    push(t);

    t->sz = 1 + (t->l ? t->l->sz : 0) + (t->r ? t->r->sz : 0);
    t->sm = t->v;
    if (t->l) {
        push(t->l);
        t->sm += t->l->sm;
    }
    if (t->r) {
        push(t->r);
        t->sm += t->r->sm;
    }
}

void rotate(Node* t) {
    Node* p = t->p;
    Node* g = p->p;
    bool is_p_root = is_root(p);

    if (!is_p_root) push(g);
    push(p);
    push(t);

    if (p->l == t) {
        p->l = t->r;
        if (p->l) p->l->p = p;
        t->r = p;
    } else {
        p->r = t->l;
        if (p->r) p->r->p = p;
        t->l = p;
    }

    p->p = t;
    t->p = g;
    pull(p);
    pull(t);

    if (!is_p_root) {
        if (g->l == p)
            g->l = t;
        else
            g->r = t;
        pull(g);
    }
}
```

```
void push_all(Node* t) {
    if (!is_root(t)) {
        push_all(t->p);
    }
    push(t);
}

Node* splay(Node* t) {
    push_all(t);
    while (!is_root(t)) {
        Node *p = t->p, *g = t->p->p;
        if (is_root(p)) {
            rotate(t);
        } else if ((g->l == p) ^ (p->l == t)) {
            rotate(t);
            rotate(t);
        } else {
            rotate(p);
            rotate(t);
        }
    }
    return t;
}

// create new tree with one node
Node* create(i64 v) { return new Node(v); }

namespace lct {
void expose(Node* v) {
    Node *last = nullptr, *cur = v;
    while (cur != nullptr) {
        splay(cur);
        cur->r = last;
        pull(cur);
        last = cur;
        cur = cur->p;
    }
    splay(v);
}

void make_root(Node* v) {
    expose(v);
    v->rev = true;
    push(v);
}

void link(Node* v, Node* u) {
    make_root(v);
    v->p = u;
}

void cut(Node* v) {
    expose(v);
    Node* u = v->l;
    if (u) {
        u->p = nullptr;
        v->l = nullptr;
        pull(v);
    }
}

void cut(Node* v, Node* u) {
    make_root(v);
    expose(u);
    v->p = nullptr;
    u->l = nullptr;
```

```
    pull(u);
}
}  // namespace lct
}  // namespace splay
```

## 1.6 Дерево отрезков с массовыми операциями RECHECK

```
/// @author Не указан в исходном файле
/// @brief Дерево отрезков с массовыми операциями RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

struct SegTree {
    int n = 0;
    vector<ll> t, c;
    static constexpr ll NEG_INF = (ll)-4e18;

    SegTree() = default;
    explicit SegTree(int n_) : n(n_), t(4 * n_, 0), c(4 * n_, 0) {}
    explicit SegTree(const vector<ll>& a) { init(a); }

    void init(int n_) {
        n = n_;
        t.assign(4 * n, 0);
        c.assign(4 * n, 0);
    }

    void init(const vector<ll>& a) {
        n = (int)a.size();
        t.assign(4 * n, 0);
        c.assign(4 * n, 0);
        build(0, 0, n, a);
    }

    void build(const vector<ll>& a) { init(a); }

    void push(int v) {
        if (c[v] != 0) {
            c[v * 2 + 1] += c[v];
            c[v * 2 + 2] += c[v];
            c[v] = 0;
        }
    }

    void pull(int v) {
        t[v] = max(t[v * 2 + 1] + c[v * 2 + 1],
                   t[v * 2 + 2] + c[v * 2 + 2]);
    }

    void build(int v, int l, int r, const vector<ll>& a) {
        if (r - l == 1) {
            t[v] = a[l];
            c[v] = 0;
            return;
        }
        int m = (l + r) / 2;
        build(v * 2 + 1, l, m, a);
        build(v * 2 + 2, m, r, a);
        t[v] = max(t[v * 2 + 1], t[v * 2 + 2]);
        c[v] = 0;
    }

    void add(int ql, int qr, ll val) { add(0, 0, n, ql, qr, val); }

    void add(int v, int l, int r, int ql, int qr, ll val) {
        if (ql <= l && r <= qr) {
            c[v] += val;
            return;
        }
        if (r <= ql || qr <= l) return;
        int m = (l + r) / 2;
        push(v);
        add(v * 2 + 1, l, m, ql, qr, val);
        add(v * 2 + 2, m, r, ql, qr, val);
        pull(v);
    }

    ll getmax(int ql, int qr) { return getmax(0, 0, n, ql, qr); }

    ll getmax(int v, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr) return t[v] + c[v];
        if (r <= ql || qr <= l) return NEG_INF;
        int m = (l + r) / 2;
        push(v);
        ll left = getmax(v * 2 + 1, l, m, ql, qr);
        ll right = getmax(v * 2 + 2, m, r, ql, qr);
        pull(v);
        return max(left, right);
    }
};
```

## 1.7 Декартово дерево (явное/неявное, sum + size) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Декартово дерево (явное/неявное, sum + size) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ull = unsigned long long;

struct treap {
    struct node {
        ll x, sum, sz; ull y;
        node *l, *r;
        node(): x(0), sum(0), sz(0), y(0), l(nullptr), r(nullptr) {}
    };

    static const int maxn = 300000 + 10;

    node pool[maxn];
    int nodesz = 0;
    mt19937_64 rng;
    node* root = nullptr;

    treap(): rng((uint64_t)chrono::steady_clock::now().time_since_epoch().count()) {}

    node* getnode(ll x = 0) {
        node* p = &pool[nodesz++];
        p->x = x; p->sum = x; p->sz = 1; p->l = p->r = nullptr; p->y = rng();
        return p;
    }
```

```
ll getsum(node* v) const { return v ? v->sum : 0; }
ll getsz(node* v)  const { return v ? v->sz  : 0; }

void updatesum(node* v) { v->sum = getsum(v->l) + getsum(v->r) + v->x; }
void updatesz(node* v)  { v->sz  = getsz(v->l)  + getsz(v->r)  + 1; }

void update(node* v) {
    if (!v) return;
    updatesz(v);
    updatesum(v);
}

// < x , >= x
void split(node* v, node*& l, node*& r, ll x) {
    if (!v) { l = r = nullptr; return; }
    if (v->x < x) {
        split(v->r, v->r, r, x);
        l = v; update(l);
    } else {
        split(v->l, l, v->l, x);
        r = v; update(r);
    }
}

void merge(node*& v, node* l, node* r) {
    if (!l || !r) { v = l ? l : r; return; }
    if (l->y > r->y) {
        merge(l->r, l->r, r);
        v = l; update(v);
    } else {
        merge(r->l, l, r->l);
        v = r; update(v);
    }
}

// [1, k] + [k + 1, ... ]
void splitk(node* v, node*& l, node*& r, ll k) {
    if (!v || k == 0) { l = nullptr; r = v; return; }
    if (k - getsz(v->l) - 1 >= 0) {
        splitk(v->r, v->r, r, k - getsz(v->l) - 1);
        l = v; update(l);
    } else {
        splitk(v->l, l, v->l, k);
        r = v; update(r);
    }
}

void insert(ll x) {
    node *t1 = nullptr, *t3 = nullptr, *t2 = getnode(x);
    split(root, t1, t3, x);
    merge(root, t1, t2);
    merge(root, root, t3);
}

void erase_value(ll x) {
    node *t1 = nullptr, *t2 = nullptr, *t3 = nullptr;
    split(root, t1, t2, x);
    split(t2, t2, t3, x + 1);
    merge(root, t1, t3);
}

int kth(int k) {
    node *t1 = nullptr, *t2 = nullptr, *t3 = nullptr;
    splitk(root, t2, t3, k);
    splitk(t2, t1, t2, k - 1);
        int res = t2->x;
        merge(root, t1, t2);
        merge(root, root, t3);
        return res;
    }

    void print_inorder(node* v) {
        if (!v) return;
        print_inorder(v->l);
        cout << v->x << ' ';
        print_inorder(v->r);
    }

    int size() const { return (int)getsz(root); }
    ll sum()  const { return getsum(root); }
};
```

## 1.8 Ordered set/map RECHECK

```
/// @author Не указан в исходном файле
/// @brief Ordered set/map RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

// g++ -std=gnu++17 -O2
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

template<class T, class cmp = less<T>>
using ordered_set = tree<T, null_type, cmp, rb_tree_tag, tree_order_statistics_node_update>;

template<class K, class V, class cmp = less<K>>
using ordered_map = tree<K, V, cmp, rb_tree_tag, tree_order_statistics_node_update>;
```

## 1.9 Дерево Фенвика RECHECK

```
/// @author Не указан в исходном файле
/// @brief Дерево Фенвика RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct Fenwick {
   int n;
   vector<ll> f;
   Fenwick() {}
   Fenwick(int n): n(n), f(n+1, 0) {}
   void add(int i, ll v){
       for(; i<=n; i+=i&-i) f[i]+=v;
   }
   ll sumPrefix(int i){
       ll r=0;
       for(; i>0; i-=i&-i) r+=f[i];
       return r;
   }
   ll sumRange(int l, int r){
       if(r<l) return 0;
       return sumPrefix(r)-sumPrefix(l-1);
   }
};
```

# 2 Строковые алгоритмы

## 2.1 Динамический Ахо — Корасик

```
/// @author egor
struct AhoCorasick {
    static const int ALPHABET = 26;
    struct Node {
        int next[ALPHABET];    // переходы автомата
        int go[ALPHABET];      // реальные переходы в боре
        int link = 0;          // суфф ссылка
        int up = 0;            // сжатая суфф ссылка (ведёт до ближайшего терминала)
        int cnt = 0;           // кол-во терминальных вершин в поддереве суф ссылок
        int pattern_ind = -1;  // индекс паттерна

        Node() {
            fill(next, next + ALPHABET, -1);
            fill(go, go + ALPHABET, -1);
        }
    };

    vector<Node> nodes;
    bool built = false;

    AhoCorasick() { clear(); }

    inline void clear() {
        nodes.clear();
        nodes.emplace_back();
        built = false;
    }

    inline int get_char(char c) { return c - 'a'; }

    void add_pattern(const string& s, int id = 0) {
        int v = 0;
        for (char c : s) {
            int x = get_char(c);
            if (nodes[v].go[x] == -1) {
                nodes[v].go[x] = nodes.size();
                nodes.emplace_back();
            }
            v = nodes[v].go[x];
        }
        nodes[v].cnt++;
        nodes[v].pattern_ind = id;
    }

    void build() {
        queue<int> q;
        // обрабатываем соседей корня
        for (int c = 0; c < ALPHABET; ++c) {
            if (nodes[0].go[c] != -1) {
                nodes[0].next[c] = nodes[0].go[c];
                q.push(nodes[0].go[c]);
            } else {
                nodes[0].next[c] = 0;
            }
        }

        while (!q.empty()) {
            int v = q.front();
            q.pop();
            int u = nodes[v].link;
            // проталкиваем терминальность
            nodes[v].up = (nodes[u].cnt > 0) ? u : nodes[u].up;
            nodes[v].cnt += nodes[u].cnt;

            for (int c = 0; c < ALPHABET; ++c) {
                if (nodes[v].go[c] != -1) {
                    // вычисляем суфф ссылку для ребёнка
                    int to = nodes[v].go[c];
                    nodes[v].next[c] = to;
                    nodes[to].link = nodes[u].next[c];
                    q.push(to);
                } else {
                    nodes[v].next[c] = nodes[u].next[c];
                }
            }
        }
        built = true;
    }

    // кол-во совпадений, заканчивающихся в текущем символе
    ll count_matches(int v) const { return nodes[v].cnt; }

    int next_state(int v, char c) { return nodes[v].next[get_char(c)]; }
};

struct DynamicAho {
    struct Layer {
        AhoCorasick aho;
        vector<string> words;

        void clear() {
            aho.clear();
            words.clear();
        }
    };
    vector<Layer> layers;  // инвариант - если layers[i] не пуст, то в нём 2^i строк

    // добавление строки: O(|S| * log N)
    void add_string(const string& s) {
        vector<string> current_words = {s};
        int i = 0;

        // Ищем свободный слой, собираем слова из занятых
        while (i < int(layers.size()) && !layers[i].words.empty()) {
            // сливаем слова на этом уровне
            current_words.insert(current_words.end(), layers[i].words.begin(),
                                 layers[i].words.end());
            layers[i].clear();
            i++;
        }

        // если вышли за пределы вектора, добавляем новый слой
        if (i == int(layers.size())) {
            layers.emplace_back();
        }

        // Строим новый слой (бит становится 1)
        layers[i].words = current_words;
        for (const string& w : layers[i].words) {
            layers[i].aho.add_pattern(w);
        }
        layers[i].aho.build();
    }

    // общее количество вхождений всех паттернов в текст O(|Text| * log N)
    ll count_total_matches(const string& text) {
```

```
        ll ans = 0;
        for (auto& layer : layers) {
            if (layer.words.empty()) continue;

            int v = 0;
            for (char c : text) {
                v = layer.aho.next_state(v, c);
                ans += layer.aho.count_matches(v);
            }
        }
        return ans;
    }
};
```

## 2.2 Суффиксный автомат RECHECK

```
/// @author Не указан в исходном файле
/// @brief Суффиксный автомат RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct sufauto {
    vector<map<char, ll>> next;
    vector<ll> len, link;

    vector<ll> cnt;

    sufauto() {
        node();
    }

    int node(){
        next.emplace_back();
        len.push_back(0);
        link.push_back(-1);
        cnt.push_back(0); // cnt[i] = size(endpos i)
        return (int)next.size() - 1;
    }

    int add(int last, char ch) {
        int new_node = node();
        int sz = len.size();
        cnt[new_node] = 1;
        link[new_node] = 0;
        len[new_node] = len[last] + 1;
        for (int p = last; p != -1; p = link[p]) {
            if (!next[p].count(ch)) {
                next[p][ch] = new_node;
                continue;
            }

            int q = next[p][ch];
            if (len[p] + 1 == len[q]) {
                link[new_node] = q;
                break;
            }

            int clone_q = node();
            link[clone_q] = link[q];
            link[q] = clone_q;
            link[new_node] = clone_q;
            len[clone_q] = len[p] + 1;
            next[clone_q] = next[q];
            for (; p != -1 && next[p][ch] == q; p = link[p]) {
                next[p][ch] = clone_q;
            }
            break;
        }

        return new_node;
    }

    void dfs_cnt(int v, vector<vector<ll>>& gr) {
        for (int u : gr[v]) dfs_cnt(u, gr);
        if (link[v] == -1) return;
        cnt[link[v]] += cnt[v];
    }


    void buildCnt() {
        int sz = len.size();
        vector<vector<ll>> gr(sz);
        for (int i = 1; i < sz; ++i) gr[link[i]].push_back(i); // дерево суф ссылок
        dfs(0, gr, dp);
    }
};
```

## 2.3 Суффиксное дерево RECHECK

```
/// @author Не указан в исходном файле
/// @brief Суффиксное дерево RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct State {
    int v;
    int pos;
    State(int v = 0, int pos = 0) : v(v), pos(pos) {}
};

struct ST {
    vector<map<char, int>> next;
    vector<int> link, par;
    vector<int> l, r;
    State ptr;
    int sz;
    string s;

    int &get(int v, char c) {
        if (!next[v].count(c)) next[v][c] = -1;
        return next[v][c];
    }

    int node() {
        next.emplace_back();
        link.emplace_back(-1);
        par.emplace_back(-1);
        l.emplace_back(0);
        r.emplace_back(0);
        return sz++;
    }

    ST(string s) : ptr(0, 0), sz(0), s(s) {
        node();
        for (int i = 0; i < (int)s.size(); ++i) add(i);
    }

    int len(int v) {
        return r[v] - l[v];
    }
```

```
    State go(State st, int from, int to) {
        while (from < to) {
            if (st.pos == len(st.v)) {
                st = State(get(st.v, s[from]), 0);
                if (st.v == -1) return st;
            } else {
                if (s[l[st.v] + st.pos] != s[from]) return {-1, -1};
                if (to - from < len(st.v) - st.pos)
                    return {st.v, st.pos + to - from};
                from += len(st.v) - st.pos;
                st.pos = len(st.v);
            }
        }
        return st;
    }

    int split(State st) {
        if (st.pos == len(st.v)) return st.v;
        if (st.pos == 0) return par[st.v];

        int cur = st.v;
        int mid = node();
        get(par[cur], s[l[cur]]) = mid;
        get(mid, s[l[cur] + st.pos]) = cur;

        l[mid] = l[cur];
        r[mid] = l[cur] + st.pos;
        l[cur] += st.pos;

        par[mid] = par[cur];
        par[cur] = mid;
        return mid;
    }

    int get_link(int v) {
        if (link[v] != -1) return link[v];
        if (par[v] == -1) return 0;
        int to = get_link(par[v]);
        return link[v] = split(go(State(to, len(to)), l[v] + (par[v] == 0), r[v]));
    }

    void add(int pos) {
        while (true) {
            State nptr = go(ptr, pos, pos + 1);
            if (nptr.v != -1) {
                ptr = nptr;
                return;
            }
            int mid = split(ptr);
            int leaf = node();
            l[leaf] = pos;
            r[leaf] = (int)s.size();
            par[leaf] = mid;
            get(mid, s[pos]) = leaf;

            ptr.v = get_link(mid);
            ptr.pos = len(ptr.v);
            if (!mid) break;
        }
    }
};
```

## 2.4 Суффиксный массив и LCP RECHECK

```
/// @author Не указан в исходном файле
/// @brief Суффиксный массив и LCP RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct SuffixArray {
    int n;
    string s;
    vector<ll> sa;
    SuffixArray(const string& _s) {
        n = _s.size();
        s = _s;
        sa = suffix_array(s);
    }
    vector<ll> sort_cyclic_shifts(const string& s) {
        int n = s.size();
        const int alphabet = 256;
        vector<ll> p(n), c(n), cnt(max(alphabet, n));
        for (unsigned char el: s) cnt[el]++;
        for (int i = 1; i < alphabet; ++i) cnt[i] += cnt[i - 1];
        for (int i = 0; i < n; ++i) p[--cnt[(unsigned char)s[i]]] = i;
        c[p[0]] = 0;
        int classes = 1;
        for (int i = 1; i < n; ++i) {
            if (s[p[i]] != s[p[i - 1]]) ++classes;
            c[p[i]] = classes - 1;
        }
        vector<ll> pn(n), cn(n);
        for (int h = 0; (1 << h) < n; ++h) {
            for (int i = 0; i < n; i++) {
                pn[i] = p[i] - (1 << h);
                if (pn[i] < 0) pn[i] += n;
            }
            fill(cnt.begin(), cnt.begin() + classes, 0);
            for (auto el: pn) cnt[c[el]]++;
            for (int i = 1; i < classes; ++i) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; --i) p[--cnt[c[pn[i]]]] = pn[i];
            cn[p[0]] = 0;
            classes = 1;
            for (int i = 1; i < n; i++) {
                pair<int,int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
                pair<int,int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
                if (cur != prev) ++classes;
                cn[p[i]] = classes - 1;
            }
            c.swap(cn);
        }
        return p;
    }
    vector<ll> suffix_array(string s) {
        s += char(0);
        vector<ll> sorted_shifts = sort_cyclic_shifts(s);
        sorted_shifts.erase(sorted_shifts.begin());
        return sorted_shifts;
    }
};

vector<ll> build_lcp(const string& s, const vector<ll>& sa) {
    int n = s.size();
    vector<ll> rank(n), lcp(n - 1);
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    int k = 0;
    for (int i = 0; i < n; ++i) {
        int r = rank[i];
```

```
        if (r == n - 1) {
            k = 0;
            continue;
        }
        int j = sa[r + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) ++k;
        lcp[r] = k;
        if (k) --k;
    }
    return lcp;
}
```

## 2.5 Ахо — Корасик RECHECK

```
/// @author Не указан в исходном файле
/// @brief Ахо — Корасик RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct Aho {
    const int A = 26;
    int node_cnt, pattern_cnt;
    vector<vector<int>> next;
    vector<vector<int>> terminals;
    vector<int> suf_link, terminal_link;

    Aho() : node_cnt(0), pattern_cnt(0) { node(); }

    int node() {
        next.emplace_back(A);
        terminals.emplace_back();
        suf_link.emplace_back();
        terminal_link.emplace_back();
        return node_cnt++;
    }

    inline int get(char c) {
        return c - 'a';
    }

    int add_pattern(const string &s) {
        int v = 0;
        for (auto c: s) {
            if (!next[v][get(c)]) next[v][get(c)] = node();
            v = next[v][get(c)];
        }
        terminals[v].emplace_back(pattern_cnt);
        return pattern_cnt++;
    }

    void build() {
        queue<int> q;
        q.push(0);
        while (!q.empty()) {
            int par = q.front();
            q.pop();
            for (int c = 0; c < A; ++c) {
                int child = next[par][c];
                if (!child) {
                    next[par][c] = next[suf_link[par]][c];
                    continue;
                }
                // set suffix link
                if (par == 0) suf_link[child] = 0;
                else suf_link[child] = next[suf_link[par]][c];
                // set terminal link
                if (terminals[suf_link[child]].empty()) terminal_link[child] =
terminal_link[suf_link[child]];
                else terminal_link[child] = suf_link[child];

                q.push(child);
            }
        }
    }

    int go(int v, char c) {
        return next[v][get(c)];
    }

    // get all matches ending at given position
    vector<int> get_matches(int v) {
        vector<int> res;
        while (v) {
            for (auto match: terminals[v]) res.emplace_back(match);
            v = terminal_link[v];
        }
        return res;
    }
};
```

## 2.6 Z-функция RECHECK

```
/// @author Не указан в исходном файле
/// @brief Z-функция RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

vector<int> zfunc(const string& s) {
    int n = s.size();
    vector<int> z(n);
    int j = 0;
    for (int i = 1; i < n; ++i) {
        z[i] = max(min(z[i - j], z[j] + j - i), 0);
        while (i + z[i] < n && s[i + z[i]] == s[z[i]]) {
            int c1 = z[i], c2 = i + z[i];
            z[i]++;
        }
        if (i + z[i] > j + z[j]) {
            j = i;
        }
    }
    z[0] = n;
    return v;
}
```

## 2.7 Префикс-функция RECHECK

```
/// @author Не указан в исходном файле
/// @brief Префикс-функция RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

vector<int> preffunc(const string& s) {
    int n = s.size();
    vector<int> p(n);

    for (int i = 1; i < n; ++i) {
        int j = p[i - 1];
        while (j != 0 && s[j] != s[i]) {
```

```
            j = p[j - 1];
        }
        if (s[j] == s[i]) {
            j++;
        }
        p[i] = j;
    }
    return p;
}
```

# 3 Геометрия

## 3.1 Точка и векторы RECHECK

```
/// @author Не указан в исходном файле
/// @brief Точка и векторы RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

const ld pi = acos(-1);
const ld eps = 1e-12;

struct point {
    ld x, y;
    point() {}
    point(ld x, ld y): x(x), y(y) {}
    point operator +(const point& a) const {
        return point(x + a.x, y + a.y);
    }
    point operator -(const point& a) const {
        return point(x - a.x, y - a.y);
    }
    ld sq2() {
        return x * x + y * y;
    }
    ld len() {
        return sqrt(sq2());
    }
    ld operator *(const point& a) {
        return x * a.x + y * a.y;
    }
    point operator *(ll val) {
        return point(x * val, y * val);
    }
    ld operator %(const point& a) { // векторное произведение
        return x * a.y - y * a.x;
    }
    bool operator ==(const point& a) const {
        return eq(x, a.x) && eq(y, a.y);
    }
    point rotate90() {
        return point(y, -x);
    }
    point rotate270() {
        return point(-y, x);
    }
    point rot(ld angle) {
        return point(x * cos(angle) - y * sin(angle), x * sin(angle) + y * cos(angle));
    }
    point norm() {
        return point(x / len(), y / len());
    }
    point resize(ld val) {
        return point(x * val, y * val);
    }
};
```

## 3.2 Выпуклая оболочка Грэхема O(NlogN) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Выпуклая оболочка Грэхема O(NlogN) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct r
{
    ld x, y;
    r(ld a, ld b) : x(a), y(b) {}
    r() : x(0), y(0) {}
    ld len() {return hypot(x, y);}
};

ld operator^(r a, r b) {return a.x * b.y - b.x * a.y;}
r operator-(r a, r b) {return {a.x - b.x, a.y - b.y};}
bool operator==(r a, r b) {return a.x == b.x && a.y == b.y;}

vector<r> graham(vector<r> points) {
    r p0(points[0].x, points[0].y);
    for (r p : points) {
        if (p.y < p0.y || p.y == p0.y && p.x < p0.x) {
            p0 = p;
        }
    }

    sort(points.begin(), points.end(), [&] (r a, r b) {
        return abs((a - p0) ^ (b - p0)) < EPS && (a - p0).len() > (b - p0).len() || ((a - p0) ^ (b -
p0)) > 0;
    });

    vector<r> hull;
    for (r p : points) {
        while (hull.size() >= 2) {
            r new_vec = p - hull.back();
            r last_vec = hull.back() - hull[hull.size() - 2];
            if ((last_vec ^ new_vec) < 0 || abs(last_vec ^ new_vec) < EPS)
                hull.pop_back();
            else
                break;
        }
        hull.push_back(p);
    }

    return hull;
}
```

# 4 Графы

## 4.1 Диниц O(V²E) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Диниц O(V²E) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;
```

```
struct Dinic {
    using ll = long long;

    struct Edge {
        int to;
        ll cap;
        int rev;
    };

    int n;
    vector<vector<Edge>> g;
    vector<int> level;
    vector<int> it;

    Dinic(int n) : n(n), g(n), level(n), it(n) {}

    void add_edge(int u, int v, ll cap) {
        Edge a{v, cap, (int)g[v].size()};
        Edge b{u, 0,   (int)g[u].size()};
        g[u].push_back(a);
        g[v].push_back(b);
    }

    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (const Edge &e : g[v]) {
                if (e.cap > 0 && level[e.to] < 0) {
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] >= 0;
    }

    ll dfs(int v, int t, ll f) {
        if (v == t) return f;
        for (int &i = it[v]; i < (int)g[v].size(); ++i) {
            Edge &e = g[v][i];
            if (e.cap > 0 && level[e.to] == level[v] + 1) {
                ll pushed = dfs(e.to, t, min(f, e.cap));
                if (pushed > 0) {
                    e.cap -= pushed;
                    g[e.to][e.rev].cap += pushed;
                    return pushed;
                }
            }
        }
        return 0;
    }

    ll max_flow(int s, int t) {
        const ll INF = (ll)4e18;
        ll flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            ll f;
            while ((f = dfs(s, t, INF)) > 0) {
                flow += f;
            }
        }
        return flow;
    }
};
```

## 4.2 Min Cost Max Flow (MCMF) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Min Cost Max Flow (MCMF) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct MCMF {
    using ll = long long;
    static const ll INF = (1LL << 61);

    struct Edge {
        int from, to;
        ll cap, cost;
        Edge(int _from, int _to, ll _cap, ll _cost)
            : from(_from), to(_to), cap(_cap), cost(_cost) {}
    };

    int n, s, t;
    ll flow, min_cost;
    vector<vector<int>> g;
    vector<Edge> e;
    vector<ll> d, potential;
    vector<int> par;
    bool neg;

    MCMF(int _n = 0) { // 0-based
        init(_n);
    }

    void init(int _n) {
        n = _n + 10;              // небольшой запас, как у тебя
        g.assign(n, {});
        e.clear();
        potential.assign(n, 0);
        neg = false;
    }

    void add_edge(int u, int v, ll cap, ll cost, bool directed = true) {
        if (cost < 0) neg = true;
        int id = (int)e.size();
        g[u].push_back(id);
        e.emplace_back(u, v, cap, cost);
        g[v].push_back(id ^ 1);
        e.emplace_back(v, u, 0, -cost);
        if (!directed)
            add_edge(v, u, cap, cost, true);
    }

    bool dijkstra() {
        par.assign(n, -1);
        d.assign(n, INF);

        using P = pair<ll, int>;
        priority_queue<P, vector<P>, greater<P>> q;

        d[s] = 0;
        q.emplace(0, s);

        while (!q.empty()) {
            auto [dist, v] = q.top();
```

```
            q.pop();
            if (dist != d[v]) continue;
            for (int id : g[v]) {
                const Edge &ed = e[id];
                if (ed.cap <= 0) continue;
                int to = ed.to;
                ll w = ed.cost + potential[v] - potential[to];
                if (d[v] + w < d[to]) {
                    d[to] = d[v] + w;
                    par[to] = id;
                    q.emplace(d[to], to);
                }
            }
        }

        if (d[t] == INF) return false;

        // стандартное обновление потенциалов
        for (int i = 0; i < n; i++) {
            if (d[i] < INF)
                potential[i] += d[i];
        }

        return true;
    }

    ll send_flow(int v, ll cur) {
        if (par[v] == -1) return cur;
        int id = par[v];
        int from = e[id].from;
        ll f = send_flow(from, min(cur, e[id].cap));
        e[id].cap     -= f;
        e[id ^ 1].cap += f;
        min_cost      += f * e[id].cost; // оригинальная стоимость
        return f;
    }

    // возвращает {maxflow, mincost}
    pair<ll, ll> solve(int _s, int _t, ll goal = INF) {
        s = _s;
        t = _t;
        flow = 0;
        min_cost = 0;
        potential.assign(n, 0);

        if (neg) {
            // Bellman–Ford для стартовых потенциалов
            d.assign(n, INF);
            d[s] = 0;
            bool relax = true;
            for (int it = 0; it < n && relax; ++it) {
                relax = false;
                for (int v = 0; v < n; ++v) {
                    if (d[v] == INF) continue;
                    for (int id : g[v]) {
                        const Edge &ed = e[id];
                        if (ed.cap <= 0) continue;
                        int to = ed.to;
                        if (d[to] > d[v] + ed.cost) {
                            d[to] = d[v] + ed.cost;
                            relax = true;
                        }
                    }
                }
            }
            for (int i = 0; i < n; i++)
                if (d[i] < INF)
                    potential[i] = d[i];
        }

        while (flow < goal && dijkstra()) {
            flow += send_flow(t, goal - flow);
        }
        return {flow, min_cost};
    }
};
```

## 4.3 Максимальное паросочетание (Кун с таймером) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Максимальное паросочетание (Кун с таймером) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

vector<int> g[N]; int mt[N];
int last_version[N]; int timer = 0;
bool try_Kuhn(int v) {
    if (last_version[v] == timer) return false;
    last_version[v] = timer;
    for (int u : g[v]) {
        if (mt[u] == -1) {
            mt[u] = v;
            return true;
        }
        if (try_Kuhn(mt[u])) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}

void solve() {
    int cnt = 0;
    fill(mt, mt + N, -1);
    for (int v = 1; v <= n; v++) {
        timer++;
        if (try_Kuhn(v)) {
            cnt++;
        }
    }
    cout << cnt << "\n";
    for (int u = 1; u <= m; u++) {
        if (mt[u] != -1) {
            // вывод -- паросочетание
            cout << mt[u] << " " << u << "\n";
        }
    }
}
```

## 4.4 HLD RECHECK

```
/// @author Не указан в исходном файле
/// @brief HLD RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

int n, q;
vector<int> g[N];
ll vertex_value[N];
int sz[N];
```

```
int par[N];

void calc_sz(int v, int p_node = 0) {
    par[v] = p_node;
    sz[v] = 1;

    int heavy_child_candidate = -1;
    int max_sz_child = 0;

    for (int i = 0; i < g[v].size(); ++i) {
        int& u = g[v][i];
        if (u == p_node) continue;

        calc_sz(u, v);
        sz[v] += sz[u];

        if (sz[u] > max_sz_child) {
            max_sz_child = sz[u];
        }
    }

    for (int i = 0; i < g[v].size(); ++i) {
        int u = g[v][i];
        if (u == p_node) continue;
        if (sz[u] == max_sz_child && max_sz_child > 0) {
            swap(g[v][0], g[v][i]);
            break;
        }
    }
    if (!g[v].empty() && g[v][0] == p_node) {
        for(int i = 1; i < g[v].size(); ++i) {
            if(g[v][i] != p_node) {
                swap(g[v][0], g[v][i]);
                break;
            }
        }
    }
}

int timer;
int tin[N];
int tout[N];
int head[N];

void mark_vertexes(int v, int p_node, bool heavy_way) {
    tin[v] = timer++;
    if (heavy_way) {
        head[v] = head[p_node];
    } else {
        head[v] = v;
    }

    if (!g[v].empty() && g[v][0] != p_node) {
        mark_vertexes(g[v][0], v, true);
    }

    for (int i = 0; i < g[v].size(); ++i) {
        int u = g[v][i];
        if (u == p_node || u == g[v][0]) continue;
        mark_vertexes(u, v, false);
    }
    tout[v] = timer;
}

class Node {
public:
    ll value;
    Node(ll x = 0) : value(x) {}
};

ostream& operator<<(ostream& out, const Node& nd) {
    out << nd.value;
    return out;
}

Node nll_node(0);

Node t[2 * N];
Node a[N + 1];

Node combine(Node l, Node r) {
    return Node(l.value + r.value);
}

void build(int v_seg, int vl, int vr) {
    if (vl + 1 == vr) {
        t[v_seg] = a[vl];
        return;
    }
    int vm = vl + (vr - vl) / 2;
    build(2 * v_seg, vl, vm);
    build(2 * v_seg + 1, vm, vr);
    t[v_seg] = combine(t[2 * v_seg], t[2 * v_seg + 1]);
}

void update(int v_seg, int vl, int vr, int index_to_update, Node new_val_node) {
    if (vl + 1 == vr) {
        t[v_seg] = new_val_node;
        return;
    }
    int vm = vl + (vr - vl) / 2;
    if (index_to_update < vm) {
        update(2 * v_seg, vl, vm, index_to_update, new_val_node);
    } else {
        update(2 * v_seg + 1, vm, vr, index_to_update, new_val_node);
    }
    t[v_seg] = combine(t[2 * v_seg], t[2 * v_seg + 1]);
}

Node get(int v_seg, int vl, int vr, int ql, int qr_inclusive) {
    if (ql > qr_inclusive) return nll_node;

    if (ql <= vl && (vr - 1) <= qr_inclusive) {
        return t[v_seg];
    }
    if ((vr - 1) < ql || qr_inclusive < vl) {
        return nll_node;
    }

    int vm = vl + (vr - vl) / 2;
    Node left_res = get(2 * v_seg, vl, vm, ql, qr_inclusive);
    Node right_res = get(2 * v_seg + 1, vm, vr, ql, qr_inclusive);
    return combine(left_res, right_res);
}

bool ancestor(int v_anc, int u_desc) {
    if (v_anc == 0) return true;
    if (u_desc == 0) return false;
    return tin[v_anc] <= tin[u_desc] && tin[u_desc] < tout[v_anc];
}

ll get_way_value(int cur_a, int cur_b) {
```

```
    ll result = 0;

    int original_a = cur_a;
    int original_b = cur_b;

    while (!ancestor(head[cur_a], original_b)) {
        if (cur_a == 0) break;
        result += get(1, 1, n + 1, tin[head[cur_a]], tin[cur_a]).value;
        cur_a = par[head[cur_a]];
    }

    while (!ancestor(head[cur_b], original_a)) {
        if (cur_b == 0) break;
        result += get(1, 1, n + 1, tin[head[cur_b]], tin[cur_b]).value;
        cur_b = par[head[cur_b]];
    }

    if (tin[cur_a] > tin[cur_b]) {
        swap(cur_a, cur_b);
    }
    result += get(1, 1, n + 1, tin[cur_a], tin[cur_b]).value;

    return result;
}

void build_hld() {
    calc_sz(1, 0);

    timer = 1;
    mark_vertexes(1, 0, false);

    for (int i = 1; i <= n; ++i) {
        a[tin[i]] = Node(vertex_value[i]);
    }
    build(1, 1, n + 1);
}

void solve() {
    cin >> n >> q;
    for (int i = 0; i < n; ++i) {
        cin >> vertex_value[i + 1];
    }

    for (int i = 0; i < n - 1; ++i) {
        int v, u;
        cin >> v >> u;
        v++; u++;
        g[v].push_back(u);
        g[u].push_back(v);
    }

    build_hld();

    while (q--) {
        int type;
        cin >> type;
        if (type == 0) {
            int ind, x;
            cin >> ind >> x;
            ind++;

            vertex_value[ind] += (ll)x;
            update(1, 1, n + 1, tin[ind], Node(vertex_value[ind]));
        } else {
            int v, u;
            cin >> v >> u;
            v++; u++;
            cout << get_way_value(v, u) << "\n";
        }
    }
}
```

## 4.5 Максимальное паросочетание (Кун, пример с main) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Максимальное паросочетание (Кун, пример с main) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

int n, k;
vector < vector<int> > g;
vector<int> mt;
vector<char> used;

bool try_kuhn (int v) {
    if (used[v])  return false;
    used[v] = true;
    for (size_t i=0; i<g[v].size(); ++i) {
        int to = g[v][i];
        if (mt[to] == -1 || try_kuhn (mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int main() {
    ... чтение графа ...

    mt.assign (k, -1);
    for (int v=0; v<n; ++v) {
        used.assign (n, false);
        try_kuhn (v);
    }

    for (int i=0; i<k; ++i)
        if (mt[i] != -1)
            printf ("%d %d\n", mt[i]+1, i+1);
}
```

# 5 Теория чисел

## 5.1 Расширенный алгоритм Евклида

```
/// @author OpenAI Codex
/// @brief Расширенный Евклид для знаковых long long.
/// @date 2026-11-07

struct ExtendedGcdResult {
    __int128 g, x, y;
};

// Возвращает g >= 0 и a*x + b*y = g. GCC/Clang с __int128.
inline ExtendedGcdResult extended_gcd(long long a, long long b) {
    if (b == 0) {
        if (a < 0) return {-static_cast<__int128>(a), -1, 0};
        return {a, 1, 0};
```

```
    }

    const __int128 wide_a = a;
    const auto [g, x, y] = extended_gcd(b, static_cast<long long>(wide_a % b));
    return {g, y, x - (wide_a / b) * y};
}
```

## 5.2 Китайская теорема об остатках

```
/// @author OpenAI Codex
/// @brief Объединение сравнений с произвольными положительными модулями.
/// @date 2026-11-07

#include <cassert>
#include <limits>

enum class CrtStatus { ok, inconsistent, overflow };

struct CrtResult {
    CrtStatus status;
    long long residue, modulus;
};

// Нужна заготовка extended_gcd. Объединяет x = a (mod m), x = b (mod n).
// Поля residue и modulus действительны при status == CrtStatus::ok.
inline CrtResult crt_merge(long long a, long long m, long long b, long long n) {
    assert(m > 0 && n > 0);
    const auto bezout = extended_gcd(m, n);
    const __int128 delta = static_cast<__int128>(b) - a;
    if (delta % bezout.g != 0) return {CrtStatus::inconsistent, 0, 0};

    const __int128 reduced_n = n / bezout.g;
    const __int128 modulus = m * reduced_n;
    if (modulus > std::numeric_limits<long long>::max()) {
        return {CrtStatus::overflow, 0, 0};
    }

    __int128 t = (delta / bezout.g % reduced_n) * bezout.x % reduced_n;
    if (t < 0) t += reduced_n;
    __int128 residue = (a + m * t) % modulus;
    if (residue < 0) residue += modulus;
    return {CrtStatus::ok, static_cast<long long>(residue), static_cast<long long>(modulus)};
}
```

## 5.3 Свёртка целых чисел через FFT

```
/// @author OpenAI Codex
/// @brief Свёртка целых чисел через комплексную FFT с округлением.
/// @date 2026-11-07

#include <cmath>
#include <complex>
#include <cstddef>
#include <numbers>
#include <stdexcept>
#include <utility>
#include <vector>

namespace fft_detail {
using Complex = std::complex<long double>;

inline void transform(std::vector<Complex>& a,
                      const std::vector<Complex>& roots, bool inverse) {
    const int n = static_cast<int>(a.size());
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n / 2;
        while (j & bit) {
            j ^= bit;
            bit /= 2;
        }
        j ^= bit;
        if (i < j) std::swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len *= 2) {
        const int half = len / 2;
        const int step = n / len;
        for (int start = 0; start < n; start += len) {
            for (int j = 0; j < half; ++j) {
                const Complex root = roots[j * step];
                const Complex u = a[start + j];
                const Complex v = a[start + j + half] * (inverse ? std::conj(root) : root);
                a[start + j] = u + v;
                a[start + j + half] = u - v;
            }
        }
    }
    if (inverse) {
        for (auto& x : a) x /= n;
    }
}
}  // namespace fft_detail

// c[k] = sum a[i]*b[j] по i+j=k. Сложность O(N log N), память O(N).
// N <= 2^20, произведение евклидовых норм a и b <= 10^12.
inline std::vector<long long> convolution_fft(const std::vector<long long>& a,
                                              const std::vector<long long>& b) {
    if (a.empty() || b.empty()) return {};
    constexpr std::size_t max_length = 1 << 20;
    if (a.size() > max_length || b.size() > max_length ||
        a.size() + b.size() - 1 > max_length) {
        throw std::length_error("convolution_fft: length exceeds 2^20");
    }
    const auto squared_norm = [](const std::vector<long long>& values) {
        long double sum = 0;
        for (long double x : values) sum += x * x;
        return sum;
    };
    if (squared_norm(a) * squared_norm(b) > 1e24L) {
        throw std::overflow_error("convolution_fft: input exceeds rounding range");
    }

    const int length = static_cast<int>(a.size() + b.size() - 1);
    int n = 1;
    while (n < length) n *= 2;
    std::vector<fft_detail::Complex> fa(n), fb(n), roots(n / 2);
    for (std::size_t i = 0; i < a.size(); ++i) fa[i] = a[i];
    for (std::size_t i = 0; i < b.size(); ++i) fb[i] = b[i];
    for (int j = 0; j < n / 2; ++j) {
        roots[j] = std::polar(1.0L, 2 * std::numbers::pi_v<long double> * j / n);
    }
    fft_detail::transform(fa, roots, false);
    fft_detail::transform(fb, roots, false);
    for (int i = 0; i < n; ++i) fa[i] *= fb[i];
    fft_detail::transform(fa, roots, true);

    std::vector<long long> result(length);
    for (int i = 0; i < length; ++i) result[i] = std::llround(fa[i].real());
    return result;
}
```

## 5.4 Линейное решето Эратосфена O(n) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Линейное решето Эратосфена O(n) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

struct linear_sieve {
    int n = 0;
    vector<int> lp, primes;

    void init(int n_) {
        n = n_;
        lp.assign(n + 1, 0);
        primes.clear();
        for (llt d = 2; d <= n; ++d) {
            if (lp[d] == 0) {
                lp[d] = d;
                primes.push_back(d);
            }
            for (ll i = 0; i < (int)primes.size() && primes[i] <= lp[d] && 1LL * primes[i] * d <= n; +
+i) {
                lp[d * primes[i]] = primes[i];
            }
        }
    }

    bool is_prime(int x) const {
        return x >= 2 && x <= n && lp[x] == x;
    }

    vector<pair<int,int>> factorize(int x) const {
        vector<pair<int,int>> res;
        while (x > 1) {
            int p = lp[x], c = 0;
            while (x % p == 0) { x /= p; ++c; }
            res.push_back({p, c});
        }
        return res;
    }
};
```

## 5.5 Проверка на простоту Миллера — Рабина RECHECK

```
/// @author Не указан в исходном файле
/// @brief Проверка на простоту Миллера — Рабина RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

// Быстрое возведение в степень по модулю с защитой от переполнения
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (__int128)res * base % mod;
        base = (__int128)base * base % mod;
        exp /= 2;
    }
    return res;
}

// Тест Миллера-Рабина
bool is_prime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    long long d = n - 1;
    while (d % 2 == 0) {
        d /= 2;
    }

    // Свидетели, достаточные для детерминированной проверки всех long long
    for (long long a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a) return true;

        long long x = power(a, d, n);
        if (x == 1 || x == n - 1) continue;

        bool composite = true;
        long long temp_d = d;
        while (temp_d * 2 < n - 1) {
            temp_d *= 2;
            x = (__int128)x * x % n;
            if (x == n - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false;
    }
    return true;
}
```

## 5.6 Длинная арифметика (ГПТ) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Длинная арифметика (ГПТ) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

#include <bits/stdc++.h>
using namespace std;

class Big {
    static constexpr int BASE = 1'000'000'000;
    vector<int> d;  // Младший блок первым. Пустой вектор означает 0.

    void trim() {
        while (!d.empty() && d.back() == 0) d.pop_back();
    }

public:
    explicit Big(unsigned long long x = 0) {
        for (; x; x /= BASE) d.push_back(x % BASE);
    }

    // s состоит из десятичных цифр. Ведущие нули допустимы.
    explicit Big(const string& s) {
        for (int r = int(s.size()); r > 0; r -= 9) {
            int l = max(0, r - 9);
            d.push_back(stoi(s.substr(l, r - l)));
        }
        trim();
    }

    friend istream& operator>>(istream& in, Big& a) {
        string s;
        if (in >> s) a = Big(s);
        return in;
```

```
    }

    friend ostream& operator<<(ostream& out, const Big& a) {
        if (a.d.empty()) return out << '0';
        out << a.d.back();
        char old_fill = out.fill('0');
        for (int i = int(a.d.size()) - 2; i >= 0; --i)
            out << setw(9) << a.d[i];
        out.fill(old_fill);
        return out;
    }

    // -1, 0 или 1 для a < b, a == b или a > b.
    friend int cmp(const Big& a, const Big& b) {
        if (a.d.size() != b.d.size())
            return a.d.size() < b.d.size() ? -1 : 1;
        for (int i = int(a.d.size()) - 1; i >= 0; --i) {
            if (a.d[i] != b.d[i]) return a.d[i] < b.d[i] ? -1 : 1;
        }
        return 0;
    }

    friend Big operator+(const Big& a, const Big& b) {
        Big c;
        int n = int(max(a.d.size(), b.d.size()));
        int carry = 0;
        for (int i = 0; i < n || carry; ++i) {
            int cur = carry;
            if (i < int(a.d.size())) cur += a.d[i];
            if (i < int(b.d.size())) cur += b.d[i];
            c.d.push_back(cur % BASE);
            carry = cur / BASE;
        }
        return c;
    }

    // Требуется a >= b.
    friend Big operator-(Big a, const Big& b) {
        assert(cmp(a, b) >= 0);
        int borrow = 0;
        for (int i = 0; i < int(a.d.size()); ++i) {
            int cur = a.d[i] - borrow;
            if (i < int(b.d.size())) cur -= b.d[i];
            borrow = cur < 0;
            a.d[i] = cur + borrow * BASE;
        }
        a.trim();
        return a;
    }

    friend Big operator*(const Big& a, const Big& b) {
        if (a.d.empty() || b.d.empty()) return Big();
        int n = int(a.d.size()), m = int(b.d.size());
        Big c;
        c.d.assign(n + m, 0);
        for (int i = 0; i < n; ++i) {
            long long carry = 0;
            for (int j = 0; j < m; ++j) {
                long long cur = c.d[i + j] + 1LL * a.d[i] * b.d[j] + carry;
                c.d[i + j] = int(cur % BASE);
                carry = cur / BASE;
            }
            c.d[i + m] = int(carry);
        }
        c.trim();
        return c;
    }

    // b > 0. Возвращает {частное, остаток}.
    pair<Big, uint32_t> divmod(uint32_t b) const {
        assert(b > 0);
        Big q;
        q.d.resize(d.size());
        uint64_t rem = 0;
        for (int i = int(d.size()) - 1; i >= 0; --i) {
            uint64_t cur = rem * BASE + d[i];
            q.d[i] = int(cur / b);
            rem = cur % b;
        }
        q.trim();
        return {move(q), uint32_t(rem)};
    }

    // b > 0. Деление столбиком с бинпоиском очередного блока частного.
    pair<Big, Big> divmod(const Big& b) const {
        assert(!b.d.empty());
        if (b.d.size() == 1) {
            auto [q, r] = divmod(uint32_t(b.d[0]));
            return {move(q), Big(r)};
        }

        Big q, r;
        q.d.resize(d.size());
        for (int i = int(d.size()) - 1; i >= 0; --i) {
            // r = r * BASE + d[i].
            if (!r.d.empty() || d[i] != 0) r.d.insert(r.d.begin(), d[i]);
            if (cmp(r, b) < 0) continue;

            int lo = 1, hi = BASE;
            while (hi - lo > 1) {
                int mid = (lo + hi) / 2;
                if (cmp(b * Big(mid), r) <= 0) lo = mid;
                else hi = mid;
            }
            q.d[i] = lo;
            r = r - b * Big(lo);
        }
        q.trim();
        return {move(q), move(r)};
    }
};
```

# 6 C++

## 6.1 Прагмы, генератор случайных чисел и замер времени (ГПТ, надо проверить) RECHECK

```
/// @author Не указан в исходном файле
/// @brief Прагмы, генератор случайных чисел и замер времени (ГПТ, надо проверить) RECHECK.
/// @date 2026-11-06
/// @note Дата переноса; код из приложенного файла без изменений.

// #pragma GCC optimize("O3")           // Доп. оптимизации и векторизация.
// #pragma GCC optimize("unroll-loops") // Разворачивание подходящих циклов.
// #pragma GCC target("avx2")           // Только если на сервере есть AVX2.

// Прагма действует только на следующий цикл.
#pragma GCC unroll 4
for (int i = 0; i < n; ++i) {
```

```
    sum += a[i];
}

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
// а уже считан. Включить таймер при сборке флагом -DLOCAL.
#ifdef LOCAL
const auto start = chrono::steady_clock::now();
#endif

ll sum = 0;
for (int x : a) {
    sum += x;  // Измеряемый код.
}

#ifdef LOCAL
const auto elapsed = chrono::steady_clock::now() - start;
const double ms = chrono::duration<double, milli>(elapsed).count();
cerr << "loop: " << ms << " ms\n";  // Вывод уже вне замера.
#endif
// g++ -DLOCAL main.cpp
```

# 7 Теоретические справки

## 7.1 Формулы, обоснования и условия применения

**Теория перед туром**

Начинайте с вопроса из карточки. Если он похож на условие задачи, проверьте, какие данные нужны методу и какие ограничения он использует.

**T01. КТО, расширенный НОД и остатки суммы цифр**

**Как найти целое число, когда известны его остатки?**

Расширенный Евклид возвращает НОД и два коэффициента. Например, `extended_gcd(30,18)` даёт `g=6, x=-1, y=2`, потому что `30·(-1)+18·2=6`. Если нужно получить справа 12, умножаем оба коэффициента на 2. Получить 7 невозможно, поскольку каждое слагаемое делится на 6.

В общем случае при `g=gcd(a,b)>0` уравнение `ax+by=c` решается, если `c` делится на `g`. Из одного решения `(x0,y0)` получаем все остальные, прибавляя к нему `t·(b/g, -a/g)` для целого `t`. Если `a=b=0`, достаточно проверить `c=0`.

**Объединение остатков.** Пусть число даёт остаток 3 при делении на 4 и остаток 5 при делении на 6. Все кандидаты из первого условия имеют вид `x=3+4t`. Подстановка во второе условие даёт `4t ≡ 2 (mod 6)`. Делим всё на 2 и получаем `2t ≡ 1 (mod 3)`, откуда `t ≡ 2 (mod 3)`. Значит, ответы — `11,23,35,…`, то есть `x ≡ 11 (mod 12)`.

Для остатков `a,b` и положительных модулей `m,n` получаем `mt ≡ b-a (mod n)`. Если `b-a` не делится на `g=gcd(m,n)`, ответа нет. Иначе делим сравнение на `g`. Коэффициент `m/g` теперь обратим по модулю `n/g`, и расширенный Евклид даёт его обратный. Итоговый модуль равен `lcm(m,n)`.

`crt_merge(3,4,5,6)` возвращает остаток 11 и модуль 12. Для нескольких условий начинаем с остатка 0 по модулю 1 и объединяем по одному. Проверяем статус после каждого шага. Несовместность остатков и слишком большой НОК имеют разные статусы. Успешный остаток лежит в `[0,modulus)`.

В основании `b` число сравнимо с суммой цифр по модулю `b-1`. Все степени `b` дают остаток 1. Если задача выдаёт суммы цифр в разных основаниях, из них можно собрать условия для КТО.

**T02. НОД как инвариант достижимости**

**Можно ли попасть в заданное состояние шагами фиксированной длины?**

При шагах `+w` и `-r` остаток по `g=gcd(w,r)` сохраняется. Из нуля можно попасть только в кратные `g`. Чтобы доказать достижимость, нужно также учесть, какие промежуточные состояния разрешены.

Пусть буфер ёмкости `L` изначально пуст. Запись добавляет `w` байт, чтение забирает `r`. При `L-w<x<r` нельзя сделать ни то ни другое. Тупик достижим ровно при `L<r+w-g`.

Если `L≥r+w-g`, в интервале тупика нет кратных `g`. Для обратного направления читаем при `x≥r`, иначе записываем. Без проверки ёмкости это прибавление `w` по модулю `r+w`. Такой цикл из нуля посещает все кратные `g` до `r+w-g`. Если ёмкость меньше максимума, прямо перед первым выходом за неё чтение невозможно, а запись не помещается.

**T03. Линейная алгебра над GF(2) и XOR-базис**

**Даны числа. Можно ли выбрать несколько из них и получить заданный XOR?**

XOR складывает биты без переносов. `1 xor 1 = 0`, `1 xor 0 = 1`. Одно и то же число, взятое дважды, исчезает из XOR. Например, `5 xor 3 = 6`, потому что `101 xor 011 = 110`.

Пусть даны `5,3,6`. Из `5` и `3` уже можно получить `6`. Значит, отдельное число `6` ничего не добавляет. Возможные результаты — `0,3,5,6`. XOR-базис хранит числа, из которых получаются все эти результаты. У сохранённых чисел разные старшие единичные биты.

Как вставить число:

1. Берём его старший единичный бит. Если в этом разряде уже есть число базиса, делаем XOR с ним. Старший бит исчезает.
2. Повторяем с тем, что осталось. Если нужная ячейка свободна, записываем остаток в неё. Если дошли до нуля, число уже выражается через базис.

Для `6 = 110` сначала используем `5 = 101`. Получаем `011`, то есть `3`. Затем XOR с `3` даёт ноль. Чтобы проверить запрос, делаем то же самое без вставок. Для `4 = 100` после XOR с `5` останется `001`. Числа со старшим битом в этом разряде нет, поэтому 4 получить нельзя.

Проверка и вставка стоят `O(B)` для `B` битов. У базиса из `r` чисел `2^r` разных результатов. Любые две разные комбинации различаются в старшем разряде, где они по-разному выбрали число базиса. Для максимума идём по битам сверху вниз и применяем XOR, если он увеличивает ответ.

**Зачем это в графе.** Построим остов неориентированной компоненты и запишем `d[v]`, XOR пути от корня до вершины. Неостовное ребро `(u,v,w)` замыкает цикл со значением `d[u] xor d[v] xor w`. Путь по остову из `s` в `t` даёт `d[s] xor d[t]`. Если разрешено проходить вершины и рёбра повторно, можно добавить к нему XOR любого набора циклов. До нужного цикла идём и возвращаемся тем же путём, поэтому эти дополнительные рёбра сокращаются. Для условия о простом пути это рассуждение неприменимо.

**Что означает GF(2).** Все значения равны 0 или 1, сложение — XOR. Если есть уравнения вроде `x xor y = 1`, их можно складывать друг с другом, убирая одинаковые переменные. Это Гаусс над GF(2). Строка `0 = 1` означает противоречие. Если осталось `f` свободных переменных, решений `2^f`.

### T04. DSU с потенциалами, small-to-large и параллельный бинарный поиск

**Добавляем рёбра. Как быстро узнать, соединены ли две вершины?**

DSU, или CHM, хранит для каждой группы представителя. Сначала все вершины отдельно. `unite(0,1)`, затем `unite(1,2)` дают одну группу из трёх вершин. Теперь `same(0,2)` вернёт true, а `size(1)` вернёт 3.

`find(v)` поднимается по родителям до корня и переподвешивает посещённые вершины прямо к корню. При объединении меньшую группу подвешиваем к большей. Вместе это даёт практически постоянное амортизированное время на операцию. Если корни уже одинаковые, `unite` возвращает false. Так можно пропускать рёбра, которые замкнули бы цикл при построении остова.

Для **DSU с XOR-потенциалами** у вершины дополнительно хранится XOR до родителя. При сжатии пути его нужно заменить на XOR до нового родителя. Если ребро `(u,v,w)` соединяет разные корни, XOR между корнями равен `pot[u] xor pot[v] xor w`, где `pot` измеряется до корня. При одинаковых корнях это значение нового цикла, которое можно добавить в базис из T03.

Если к группам приписаны списки, переносим меньший список в больший. Размер группы каждого перенесённого элемента хотя бы удваивается. Поэтому элемент переезжает не больше `O(log n)` раз.

Для многих запросов о минимальном пороге можно проверять их бинарные поиски вместе. Например, если разрешать рёбра с весом до `W`, связность при росте `W` сохраняется. В одном раунде сортируем середины запросов и добавляем рёбра по возрастанию веса. Все рёбра веса `W` добавляем до ответа на запрос с этим порогом.

### T05. Граф состояний, SCC, DAG и потенциалы

**Даны зависимости. В каком порядке их обработать и есть ли цикл?**

Пока DFS обрабатывает вершину, она лежит в стеке. Ребро в вершину, которая всё ещё в стеке, замыкает цикл. Если циклов нет, топологический порядок ставит начало каждого ребра раньше его конца. Когда `v` зависит от `u`, это позволяет посчитать `u` до `v`.

В компоненте сильной связности (SCC) из каждой вершины можно дойти в каждую другую по направлению рёбер. Если заменить каждую SCC одной вершиной, получится ациклический граф. SCC находятся за `O(V+E)`.

Для равенств `x[v]-x[u]=w` можно выбрать одну величину равной нулю и восстановить остальные обходом. Например, из `B-A=2` и `C-B=3` следует `C-A=5`. Если ещё дано `C-A=4`, обнаружится противоречие.

### T06. Кратчайшие пути и сжатие плотных переходов

**Как представить переходы и какой алгоритм выбрать по их ценам?**

Единичные цены — BFS. Только 0 и 1 — 0–1 BFS. Неотрицательные цены — Дейкстра. Если граф ациклический, достаточно DP в топологическом порядке, в том числе для отрицательных цен.

Для пути можно хранить пару `(пересадки, время)`. Сравниваем сначала пересадки, при равенстве время. Дейкстра работает с таким порядком для неотрицательных прибавок.

Если из любого терминала аэропорта можно доехать в любой другой за одну пересадку, добавим узел «автобус». Из каждого терминала вход в автобус стоит одну пересадку, а из автобуса выход в любой терминал бесплатен. Вместо $k^2$ переходов храним `2k` дуг. Узел можно использовать, когда все получившиеся пары переходов действительно разрешены.

### T07. Паросочетания, теорема Холла и отсечение кандидатов

**Каждому участнику нужен свой предмет из списка допустимых. Можно ли раздать?**

Строим граф из участников и предметов. Ребро означает, что предмет подходит участнику. Паросочетание выбирает рёбра без общих концов. Например, если трём людям подходят только два предмета, раздать всем невозможно. По теореме Холла

это единственная форма препятствия. У любой группы из k участников должно быть не меньше k доступных ей предметов вместе.

Если есть цены, можно искать распределение потоком минимальной стоимости. При цене, зависящей только от предмета, для n участников хватит до n самых дешёвых вариантов на каждого. Даже если участник получил отброшенный дорогой предмет, среди оставленных n найдётся свободный. Другие участники занимают не больше n-1 предметов.

**T08. Потоки, ограничения по группам и разрезы**
**Как распределить заявки, когда у каждого пути есть несколько лимитов?**

Ребро с ёмкостью 5 пропускает не больше пяти единиц. В промежуточной вершине сколько вошло, столько и выходит. Чтобы ограничить саму вершину, разделяем её на вход и выход и ставим ёмкость между ними.

Например, склад принимает до 10 заказов, из них автомобильных до 3. Автомобильные сначала идут через дугу с ёмкостью 3. После неё они попадают в общий узел склада вместе с остальными. Из общего узла выходит одна дуга с ёмкостью 10. Так соблюдаются оба лимита.

Максимальный поток равен ёмкости минимального разреза. После поиска потока вершины, куда ещё можно добраться от источника по остаточным дугам, образуют его сторону разреза. При минимизации стоимости обратная дуга имеет противоположную цену. Если внутри алгоритма используется Дейкстра, потенциалы должны делать приведённые цены `cost(u,v)+pot[u]-pot[v]` неотрицательными на доступных дугах.

**T09. Деревья: Euler tour, HLD, расстояния и центроиды**
**Нужны запросы на поддереве, пути или по расстоянию?**

При DFS пронумеруем вершины на входе. Обход полностью заканчивает поддерево перед переходом в соседнюю ветку, поэтому поддерево — отрезок `[tin[v],tout[v])`. Теперь прибавление ко всему поддереву становится прибавлением на отрезке массива.

LCA — ближайший общий предок двух вершин. Путь между вершинами идёт через него. Поэтому расстояние равно `depth[u]+depth[v]-2·depth[lca]`. Для взвешенного дерева берём суммы весов от корня вместо глубин.

HLD выбирает у каждой вершины самого большого ребёнка по размеру поддерева. Остальные переходы уменьшают размер хотя бы вдвое. Поэтому путь разбивается на `O(log n)` отрезков тяжёлых цепей, которые можно запросить деревом отрезков.

Центр минимизирует наибольшее расстояние до других вершин. Центроид делит дерево на куски размера не больше половины. При рекурсивном разбиении по центроидам у вершины `O(log n)` предков. Для запросов по расстоянию храним счётчики относительно каждого центроида и вычитаем вклад ветви запроса, чтобы избежать двойного учёта.

**T10. Дерево отрезков как моноид; ленивые операции**
**Что хранить на отрезке, чтобы собрать ответ из двух половин?**

Для суммы достаточно числа. Для чередующейся суммы нужны ещё знаки. Если левая часть нечётной длины, правая начинается с минуса. Храним `(s,p)`, где `s` — чередующаяся сумма, `p` — чётность длины. Тогда слияние `(s,p)` и `(t,q)` даёт `(s + (-1)^p·t, p xor q)`. Пустой отрезок даёт `(0,0)`. При фильтрации считаем чётность количества оставшихся элементов.

Ленивая операция должна уметь обновить итог целого отрезка. Например, `x→ax+b` меняет сумму на `a·sum+b·len`. Если сначала применили `(a1,b1)`, затем `(a2,b2)`, общий тег равен `(a2a1,a2b1+b2)`. Порядок важен. Для умножения значений и запросов суммы нужен соответствующий тег и агрегат.

Если элемент действительно растёт хотя бы вдвое и никогда не превышает `A`, его можно явно обработать `O(log A)` раз до следующего присваивания. Умножение на 1 можно пропустить.

**T11. Неявный treap и амортизация уменьшением вдвое**
**Нужно вставлять, вырезать или разворачивать куски последовательности?**

У treap по позиции номер элемента вычисляется из размера левого поддерева. `split(t,k)` отделяет первые `k` элементов, `merge(a,b)` склеивает две последовательности. Чтобы вырезать `[l,r)`, сначала разрезаем по `l`, затем правый кусок по `r-l`. При случайных приоритетах операции занимают ожидаемые `O(log n)`.

Разворот меняет детей местами и ставит флаг. При спуске передаём его детям, а после изменения детей пересчитываем размер и агрегат. Если агрегат зависит от порядка, храним итог в обеих ориентациях.

Если каждая дорогая пересборка оставляет не больше половины целого невосполняемого запаса `H`, пересборок будет `O(log(H+1))`.

**T12. DP с малой памятью и восстановлением ответа**
**Ответ надо вывести целиком, а таблица DP не помещается?**

Храним только нужные слои DP. Отдельно для каждого состояния можем записать, откуда пришли. Например, в клетку можно войти сверху или слева. Достаточно сохранить направление и потом пройти назад от финиша. Для трёх типов переходов хватает двух бит на состояние, если по типу можно однозначно вычислить предка.

Другой способ — посчитать стоимость до середины с обоих концов, выбрать точку встречи оптимального пути и повторить для двух половин. Это снижает память. Время нужно оценить по сумме размеров всех подзадач.

### T13. Вероятность: индикаторы, первый шаг и самопереходы

**Нужно среднее число событий или среднее время до результата?**

Если требуется среднее число совпадений, каждому совпадению сопоставляем 0 или 1. Его ожидание — вероятность совпадения. Складываем эти вероятности. Линейность ожидания работает и для зависимых событий.

Если один шаг стоит одну единицу времени, пишем `E[s]=1+Σ P(s→t)·E[t]`. Например, бросаем кубик до шестёрки. На неудаче всё начинается заново, поэтому `E=1+(5/6)E` и `E=6`. В общем случае самопереход с вероятностью `q` переносим влево и делим на `1-q`. Когда состояния зависят друг от друга по циклу, нужны совместные уравнения.

Для случайного удаления из очереди можно идти назад. Если перед удалением в момент `t` есть `k` заявок, одна выбранная заявка уйдёт сейчас с вероятностью `1/k`. Поэтому её ожидаемый момент ухода обновляется как `F ← t/k+(k-1)F/k`.

По модулю дробь `a/b` считаем как `a·b^(-1)`, если `b` обратимо. Для усреднений по двум равновероятным исходам глубины до `D` можно хранить числитель в масштабе `2^D`.

### T14. Факториалы, мультиномиальные коэффициенты и циклы перестановки

**Сколько разных порядков, если часть элементов одинаковые?**

Для `L` элементов с частотами `c[i]` ответ `L! / ∏c[i]!`. Для цифр `0,1,1` это три порядка. Чтобы убрать ведущий ноль, фиксируем один ноль первым и вычитаем число порядков оставшихся цифр. Здесь остаются `101` и `110`. Для вычислений через `fact` и `invfact` по простому модулю `p` нужно `L<p`.

Если считаем перестановки по циклам, новый элемент либо образует новый цикл, либо вставляется после одного из старых `k` элементов. Поэтому `c(k+1,j)=c(k,j-1)+k·c(k,j)`, где `j` — число циклов. Эти числа также получаются как коэффициенты `x(x+1)…(x+k-1)`.

### T15. Свёртка целых чисел через FFT

**Нужно сразу узнать, сколько пар дают каждую возможную сумму?**

Пусть `a[i]` — число предметов веса `i` в первой группе, а `b[j]` — во второй. Для суммы `k` подходят пары весов `i+j=k`. Каждая такая пара весов даёт `a[i]·b[j]` вариантов. Сложим их и получим `c[k]`. Массив всех этих ответов называется **свёрткой**.

Например, `a=[2,1]` и `b=[3,4]`. Сумму 0 дают только веса 0 и 0, получается 6 пар. Сумму 1 дают веса 0 и 1 либо 1 и 0, получается `2·4+1·3=11`. Для суммы 2 получаем 4 пары. Итого `[6,11,4]`.

Ту же операцию видно в $(2+x)(3+4x)=6+11x+4x^2$. Степени складываются как веса. Значит, свёртку можно получить умножением многочленов.

**Что ускоряет FFT.** Вместо попарного умножения коэффициентов вычисляем значения обоих многочленов в одинаковых точках. В каждой точке значение произведения — просто произведение двух чисел. Затем обратная FFT восстанавливает коэффициенты. Специальный выбор точек на единичной окружности позволяет делить задачу на чётные и нечётные степени и выполнять преобразование за `O(N log N)`.

`convolution_fft(a,b)` делает оба преобразования и округление. На вход подаются два `vector<long long>`, на выходе коэффициенты. Пустой вход даёт пустой результат. Длину FFT округляем вверх до степени двойки, не меньшей `a.size()+b.size()-1`. Иначе старшие коэффициенты наложатся на младшие. Чтобы посчитать разности, разворачиваем второй массив. Разность `d` окажется по индексу `d+b.size()-1`.

Заготовка считает в `long double` и округляет вещественную часть. Она принимает `N≤2^20` и произведение норм входов до `10^12`. Норма массива — корень из суммы квадратов. При выходе за эти пределы функция сообщает об ошибке. Для больших точных коэффициентов можно использовать свёртки по нескольким простым модулям и восстановление КТО.

### T16. Неравенства, транспорт на прямой и допустимые суммы

**Как распределить общий ресурс или перенести предметы между соседями?**

Если на рёбрах пути надо распределить время `T`, а штраф равен `Σ c[i]·l[i]/t[i]`, то неравенство Коши–Буняковского даёт минимум `(Σ sqrt(c[i]·l[i]))²/T`. Время на ребре должно быть пропорционально `sqrt(c[i]·l[i])`. Значит, путь можно искать с такими весами рёбер.

Пусть в двух массивах одинаковое общее число предметов. При соседних переносах через границу после `i` необходимо перенести её дисбаланс `|A[i]-B[i]|`, где `A,B` — префиксные суммы. Этого количества достаточно. Складываем стоимости границ. Если нужен общий ответ для нескольких распределений, берём медиану префиксных сумм в каждой позиции. Эти медианы не убывают, поэтому разности соседних префиксов допустимы.

Если соседние целые значения отличаются максимум на 1 и значения неотрицательны, фиксированная точка `t[a]=A` даёт в позиции `i` границы `max(0,A-|i-a|) ≤ t[i] ≤ A+|i-a|`. Для двух фиксированных точек пересекаем

их ограничения. Суммы минимального и максимального профилей можно посчитать по арифметическим прогрессиям.

**T17. Строки: границы, подстроки и палиндромы**
**Нужно найти вхождения или понять, где совпадают куски строки?**

У строки ababa собственные границы — a и aba. Это куски, которые одновременно стоят в начале и в конце. Префикс-функция хранит длину наибольшей собственной границы каждого префикса. Из длины j переходим к следующей меньшей через pi[j-1]. На этих переходах строится KMP. Z-функция отвечает на другой вопрос. Сколько букв с позиции i совпадают с началом строки?

Переходы j→pi[j-1] образуют дерево. Длины префиксов образца, совпадающих с концом прочитанного текста, лежат на пути от состояния KMP к корню. Если пронумеровать дерево обходом, проверка предка становится проверкой попадания в отрезок.

Для числа различных подстрок суффиксный автомат даёт сумму len[v]-len[link[v]] по состояниям кроме корня. Суффиксный массив даёт n(n+1)/2-ΣLCP. Общий префикс двух суффиксов — минимум LCP между их рангами.

Манакер считает радиусы палиндромов около всех центров за O(n). В бинарном чётном антипалиндроме симметричные биты различны. Если инвертировать биты на нечётных позициях, они станут равны. Дальше можно применить обычного Манакера для чётных палиндромов.

**T18. Геометрия: ориентация, отражения, оболочки и сечения**
**По какую сторону от прямой лежит точка и где экстремум?**

Знак cross(b-a,c-a) показывает, куда повернули из a→b в a→c. Положительный знак — влево, отрицательный — вправо, ноль — точки на одной прямой. Для отражения точки p сначала находим её проекцию h на прямую, затем берём 2h-p. Для прямой через a с направлением d проекция равна a+d·dot(p-a,d)/dot(d,d).

Чтобы найти максимум проекции точек на направление d, достаточно выпуклой оболочки. Линейная функция внутри оболочки принимает значение между значениями на её вершинах.

Площадь можно считать по вертикальным сечениям. Если суммарная площадь треугольников меньше площади прямоугольника, найдётся вертикаль, где сумма длин сечений меньше его высоты. На этой вертикали точно останется непокрытая точка. Для треугольника длина сечения между соседними абсциссами вершин меняется линейно.

**T20. Конечные игры и малые грамматики**
**Можно ли описать все положения или все допустимые слова?**

Если ходов больше нет, текущий игрок проиграл. Положение выигрышно, когда можно передать сопернику проигрышное. Например, если за ход можно взять одну или две монеты, 1 и 2 монеты — выигрышные состояния. При трёх монетах оба хода дают сопернику выигрышное состояние, поэтому 3 — проигрышное. Так считаем от меньших состояний к большим.

Если допустимых слов мало, можно сгенерировать весь словарь. Канонических римских записей от 1 до 3999 всего 3999. Для разбиения строки проверяем, какое слово заканчивается в текущей позиции, и переходим из DP по префиксу перед этим словом.

**T21. Интерактив: информация, устойчивые сравнения и бюджет**
**Сколько ответов может дать запрос и хватает ли лимита?**

Чтобы различить восемь возможных чисел вопросами с ответом «да» или «нет», нужно хотя бы три вопроса. После двух вопросов есть только четыре возможных последовательности ответов. В общем случае при M кандидатах и q≥2 ответах граница — ceil(log_q M). Она может не достигаться, если разрешённые вопросы плохо делят кандидатов.

Перед реализацией выясняем, ограничено число запросов или их общий размер. Проверяем равенства и какой ответ означает остановку. Каждый запрос заканчиваем переводом строки и flush.

**T24. Арифметические функции, факторизация и массовая обработка**
**Как посчитать делители после разложения на простые?**

У 12=2²·3 есть три варианта степени двойки и два варианта степени тройки. Значит, делителей шесть, а их сумма (1+2+4)(1+3)=28. Для n=∏p[i]^alpha[i] действуем так же. Количество — произведение alpha[i]+1, сумма — произведение сумм степеней каждого простого.

В n! степень простого p равна floor(n/p)+floor(n/p²)+…. Сначала считаем кратные p, потом добавляем вторые множители из кратных p² и так далее. Для двойного факториала (2t)!!=2^t·t!.

При переборе делителей парами d,n/d квадратный корень считаем один раз. Чтобы посчитать пары с суммой, делящейся на d, группируем числа по остаткам. К остатку r подходит -r mod d. Если остаток совпадает со своим партнёром, выбираем два разных элемента из одной группы. Miller–Rabin отвечает на вопрос о простоте. Для получения простых множителей нужен алгоритм факторизации.

# 8 Идеи решения задач

## 8.1 Повторяющиеся приёмы московских контестов

**Как узнать идею в новом условии**

В каждой карточке есть признак задачи и маленький пример. Сначала сравните ограничения примера с теми, что даны на туре.

**Мало разных значений**

Дан длинный массив, но все числа от 0 до 10. Нужно посчитать пары индексов с суммой `S`. Запишем 11 частот. К значению `x` подходит `S-x`. Перемножаем частоты при `x < S-x`, чтобы учесть пару один раз. При `x = S-x` выбираем два индекса из одной группы. Длинный массив прочитали один раз, дальше работаем только с частотами.

**Круг отличается от линии одним стыком**

Люди стоят по кругу, у каждого мешок разных чисел. Каждый отдаёт одно своё число вправо и получает число слева. Хотим, чтобы в мешках не было повторов. Если полученное число изначально было в мешке, человек обязан отдать именно его. Иначе он может отдать любое своё.

Пройдём по линии, начиная с произвольного выбора первого человека. Теперь нужно проверить только число, которое последний передаёт первому. Если оно создаёт повтор, протянем его дальше по начальному участку круга, пока оно есть в мешках. В первом мешке без этого числа старый выбор снова подходит. Если число есть у всех, его передают все.

**Достаточно пары с нужным свойством**

Нужно поставить точку посередине между двумя участниками так, чтобы остальные не пришли раньше этой пары. Кажется, что надо найти глобально ближайшую пару. Хватит взять любого участника `A` и найти ближайшего к нему `B`. Пусть расстояние между ними `d`. В середине отрезка до обоих `d/2`. Если кто-то оказался бы ближе к середине, он находился бы ближе `d` к `A`, что противоречит выбору `B`. Поиск соседа фиксированной точки требует одного прохода.

**Ожидаемая стоимость складывается из вкладов**

Задача просит среднюю стоимость случайного процесса. Сначала выпишите цену обязательных действий. Затем найдите, какой отдельный факт добавляет единицу к стоимости.

Например, обработка равномерно случайной перестановки стоит `n` действий плюс одно за каждую инверсию. У каждой пары чисел вероятность быть в обратном порядке равна `1/2`. Средняя стоимость равна `n + n(n-1)/4`. Для сложения ожиданий связь между парами не мешает.

**Жадный выбор можно обосновать обменом**

Нужно взять как можно больше непересекающихся встреч. Выбираем встречу с самым ранним концом. Возьмём любой оптимальный набор и заменим в нём первую встречу нашей. Она заканчивается не позже, поэтому остальные встречи остаются допустимыми. Теперь можно повторить тот же аргумент для оставшегося времени.

**Будущему нужна только часть истории**

Из ряда чисел нужно выбрать набор с максимальной суммой, причём три выбранных числа подряд запрещены. При движении слева направо будущим решениям важно только, сколько последних чисел мы взяли подряд. Храним три значения DP для длин 0, 1 и 2. Пропуск сбрасывает длину в ноль, выбор увеличивает её на один. Из состояния 2 выбирать следующее число уже нельзя.

**Проверка всего отрезка сводится к краям диапазона**

Есть обновления массива и запрос «лежат ли все числа этого отрезка в диапазоне `[L,R])?». Хватит минимума и максимума отрезка. Если минимум хотя бы L, а максимум строго меньше R`, все элементы подходят. Пара из минимума и максимума сливается из двух половин, поэтому её удобно хранить в дереве отрезков.

**Считать вклад элемента**

Нужно сложить суммы всех подотрезков. Вместо перебора отрезков посмотрим на элемент в позиции `i`. Слева у него `i+1` возможных начал, справа `n-i` возможных концов. Он войдёт в ответ `(i+1)(n-i)` раз. Складываем `a[i]·(i+1)(n-i)` за один проход.

**Вычесть плохие варианты без повторов**

Нужно число подотрезков хотя бы с одним нулём. Всего есть `n(n+1)/2` подотрезков. Плохие целиком лежат в непрерывных блоках между нулями. Блок длины `k` даёт `k(k+1)/2` плохих вариантов. Каждый плохой отрезок лежит ровно в одном блоке, поэтому можно просто вычесть сумму по блокам.

**События одной группы читать из старого слоя**

Можно пользоваться переходами только со строго возрастающим временем. Если рёбра `1→2` и `2→3` имеют время 5, по ним нельзя пройти подряд. При немедленном обновлении DP второе ребро увидит результат первого и ошибочно разрешит путь. Все переходы времени 5 считаем по старым значениям, затем применяем их одновременно.

**Хранить общий сдвиг отдельно**

Все точки часто двигаются на одинаковое расстояние, а иногда надо узнать их положение. Храним исходную координату и общий `shift`. Сдвиг на 7 означает

только `shift += 7`. Если теперь надо найти точки с реальной координатой хотя бы 15, ищем в сохранённых координатах начиная с `15-shift`.

**Идти справа налево, если спрашивают о будущем**

Для каждого элемента надо найти следующее вхождение того же числа. Идём справа налево. В словаре хранится ближайшая встреченная позиция каждого значения. Сначала читаем её как ответ, потом записываем текущую. Для `[2,1,2,2]` у первой двойки ответ — третья позиция, у третьей — четвёртая. Поиск по оставшемуся хвосту не нужен.

**Раскрыть максимум до выбора структуры**

В переходе DP встречается `max_j(dp[j]+max(a[j],x))`. У каждого `j` можно взять либо `dp[j]+a[j]`, либо `dp[j]+x`. Значит, ответ — максимум из `max_j(dp[j]+a[j])` и `x+max_j(dp[j])`. Если допустимые `j` добавляются по одному, поддерживаем два числа.

**Искать среди остатков вместо целых чисел**

Нужно найти наименьшее положительное число из цифр 0 и 1, кратное `m`. Строим его слева направо. Для продолжения важен только остаток `r`. После добавления цифры `d` новый остаток равен `(10r+d) mod m`. Запускаем BFS от числа 1, переходы пробуем в порядке 0, затем 1. Когда получим остаток 0, восстанавливаем цифры по предкам. У графа всего `m` состояний, даже если само число очень длинное.

**Сначала доказать границу, затем построить ответ**

Нужно как можно меньше рёбер в связном простом графе, где одно ребро входит в два простых цикла. При `n-1` рёбрах граф — дерево, при `n` есть только один цикл. Значит, нужно хотя бы `n+1`. Для `n≥4` строим два треугольника с общим ребром, затем подвешиваем оставшиеся вершины листьями. Получается ровно `n+1` рёбер.

**Перебрать маленькую часть полностью**

После основной конструкции осталось решить, какие из пяти объектов взять. Можно попробовать все 32 подмножества и выбрать подходящее. Надо доказать, что осталось не больше пяти объектов и что проверка подмножества учитывает все ограничения. Тогда малый перебор становится частью общего алгоритма.